



Modular Verification of Op-Based CRDTs in Separation Logic

ABEL NIETO, Aarhus University, Denmark
LÉON GONDELMAN, Aarhus University, Denmark
ALBAN REYNAUD, ENS Lyon, France
AMIN TIMANY, Aarhus University, Denmark
LARS BIRKEDAL, Aarhus University, Denmark

Operation-based Conflict-free Replicated Data Types (op-based CRDTs) are a family of distributed data structures where all operations are designed to commute, so that replica states eventually converge. Additionally, op-based CRDTs require that operations be propagated between replicas in causal order. This paper presents a framework for verifying safety properties of CRDT implementations using separation logic. The framework consists of two libraries. One implements a Reliable Causal Broadcast (RCB) protocol so that replicas can exchange messages in causal order. A second OpLib library then uses RCB to simplify the creation and correctness proofs of op-based CRDTs. OpLib allows clients to implement new CRDTs as purely-functional data structures, without having to reason about network operations, concurrency control and mutable state, and without having to each time re-implement causal broadcast. Using OpLib, we have implemented 12 example CRDTs from the literature, including multiple versions of replicated registers and sets, two CRDT combinators for products and maps, and two example use cases of the map combinator. Our proofs are conducted in the Aneris distributed separation logic and are formalized in Coq. Our technique is the first work on verification of op-based CRDTs that satisfies both of the following properties: it is *modular* and targets executable *implementations*, as opposed to high-level protocols.

CCS Concepts: • **Theory of computation → Program verification**; **Distributed algorithms**; **Separation logic**.

Additional Key Words and Phrases: separation logic, distributed systems, CRDT, replicated data type, formal verification, causal broadcast

ACM Reference Format:
Abel Nieto, Léon Gondelman, Alban Reynaud, Amin Timany, and Lars Birkedal. 2022. Modular Verification of Op-Based CRDTs in Separation Logic. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 188 (October 2022), 29 pages. https://doi.org/10.1145/3563351

1 INTRODUCTION

To an outside observer, a distributed system ideally appears to function as a single computer, and the fact that the system is composed of multiple collaborating processes is an implementation detail hidden inside the proverbial black box. This behaviour is formally captured by the notion of *linearizability* Herlihy and Wing [1990], which says that concurrent execution histories of a linearizable data structure can be re-ordered so that operations appear to take place (a) atomically and (b) in a manner that is consistent with sequential order.

Authors' addresses: Abel Nieto, Aarhus University, Denmark, abeln@cs.au.dk; Léon Gondelman, Aarhus University, Denmark, gondelman@cs.au.dk; Alban Reynaud, ENS Lyon, France, alban.reynaud@ens-lyon.fr; Amin Timany, Aarhus University, Denmark, timany@cs.au.dk; Lars Birkedal, Aarhus University, Denmark, birkedal@cs.au.dk.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2022 Copyright held by the owner/author(s).
2475-1421/2022/10-ART188
https://doi.org/10.1145/3563351

Alas, the CAP[1] theorem [Gilbert and Lynch 2002] shows that, in the presence of network partitions, a system can be either linearizable or available, but not both. *Available* in this context means that the nodes in different network partitions can (independently) continue to service client requests, without waiting for the partitions to heal.

Confronted with this consistency vs availability dilemma, practitioners have developed systems that trade off stronger forms of consistency (e.g. linearizability and sequential consistency) in favour of better availability (e.g. [Bailis et al. 2013; Chang et al. 2008; Chodorow and Dirolf 2010; Lloyd et al. 2011; Sivasubramanian 2012; Tyulenev et al. 2019]). This is possible by adopting weaker consistency models; among such models are *strong eventual consistency* (SEC) [Shapiro et al. 2011b] and *causal consistency* [Ahamad et al. 1995]. For example, in SEC two processes that read from a replicated register might observe different values even though no intervening writes have occurred locally (something not possible when reading from sequentially-consistent local memory from within a process). Eventually, however, the state of the replicated register at different replicas must converge. More precisely, SEC requires the following two properties (note the first is a liveness property while the latter is a safety property):

- *(Eventual Delivery)* An update delivered to a correct replica is eventually delivered to all replicas.
- *(Convergence)* Replicas that have delivered the same updates eventually reach equivalent states.

Conflict-free Replicated Datatypes (CRDTs) [Shapiro et al. 2011a] are a class of distributed systems where a data structure (e.g. register, set, or map) is replicated over multiple replicas that mutate its state via local operations. Because replicas are allowed to invoke operations without coordinating with others, different replicas might arrive at conflicting states. CRDTs resolve such conflicts automatically. There are two main ways of going about this. One option is to model the replica state as a (join) semilattice, so that merges are accomplished by taking least upper bounds (joins); these are *state-based* or *convergent* CRDTs. Changes are then propagated by sending the entire state to other replicas on the (possibly unreliable) network. Another option is to propagate, instead of the entire state, just the effect of each individual update. It becomes then necessary to enforce that each operation is executed exactly once (at most once for the convergence and at least one for the eventual delivery properties above), which typically requires broadcasting primitives that offer reliable delivery. Furthermore, it is also necessary to enforce that some or all operations commute so that concurrent operations can be applied in any order. This last class, known as operation-based (*op-based*) or *commutative* CRDTs, is the focus of this paper. [2]

Consider the following example of a counter data structure replicated over two nodes A and B:

```
(* Node A *)                (* Node B *)
add 1; add 200              add 2; let v = read () in assert((v = 2) || (v = 3) || (v = 203))
```

The counter exports two operations: `add(z)`, which adds an integer `z` to the counter, and `read()`, which returns the counter's current value. This CRDT is known as a *positive-negative counter* (PN-Counter)[Shapiro et al. 2011a].

One question of interest for the example above is what are the possible values of `v`. Because the counter should remain available even if A and B are partitioned, A's `add(1)` should execute without trying to synchronize with B. This means that A's and B's `add` operations potentially happen concurrently. By contrast, when A's two operations are broadcast to B, they should be applied by B following A's program order. Finally, when B reads, we do not know whether A's updates have been received, but we do expect that the `add(2)` has been recorded locally. This means that

[1] Consistency, Availability, Partition tolerance

[2] From now on whenever we use the term *CRDT* the reader can safely assume that we mean *op-based* CRDT, unless explicitly noted otherwise.

the possible values for `v` are `2` (only the local `add` has been applied), `3` (only A's first `add` has been applied), and `203` (all three `add`s have been applied). Results like `0`, `200` and `202` are not valid answers.

Causal Delivery. Our intuitions about valid execution traces in the example above can be captured by a *happens-before* or *causality* relation on events [Lamport 1978]. Let a and b be two events (possibly taking place at different processes). Then a *happens before* b (and b is *causally dependent* on a), written $a \longrightarrow b$, if one of the following holds:

- a and b take place in the same process, and $a < b$ according to *program order*.
- a is the event of sending a message m and b is the corresponding event where m is received.
- $a \longrightarrow c$ and $c \longrightarrow b$ for some other event c (the transitive closure of the above two rules).

If neither $a \longrightarrow b$ nor $b \longrightarrow a$, then we say they are *concurrent*, written $a||b$. Informally, we say that events are *causally delivered* if the following property holds: if an event e is delivered[3] to a replica p, then all events on which e causally depends must have been previously delivered to p. We can then require that valid PN-counter execution traces satisfy causal delivery of operations. Indeed, this is a common requirement for many CRDTs in the literature [Baquero et al. 2014].

Reliable Causal Broadcast. One way to realize the guarantees of causal delivery is to implement a one-to-many communication protocol known as *Reliable Causal Broadcast* (RCB) [Cachin et al. 2011]. In RCB, a group of N replicas send each other messages. The protocol's interface consists of two functions: broadcast(msg), which sends message msg to all other $N-1$ replicas, and deliver(), which returns a received message (if one exists) while respecting causal order.

Verifying CRDTs. Because CRDTs are data structures replicated across multiple processes, each of which is allowed to reorder concurrent operations, they are challenging to specify and verify.

The main property of interest for verification is SEC [Shapiro et al. 2011b] which as we mentioned can be divided into convergence and eventual delivery. [4] However, convergence does not say how the CRDT's final state is computed from the set of received operations. Burckhardt et al. [2014] addressed this question by showing how to give *functional correctness* specifications for CRDTs. Another consideration is whether the verified properties can be reused by components other than the CRDT: that is, whether the verification technique is *modular*. The recent work of Liang and Feng [2021] presents the first modular verification technique for op-based CRDTs.

An additional design decision is the level of detail at which to model the CRDT that is the target of verification. There are roughly two options: one can model the CRDT as a high-level protocol, perhaps assuming that the network is reliable or ignoring node-local concurrency. Alternatively, we can implement the CRDT in a general-purpose programming language where we have to deal with a plethora of low-level (but realistic) details such as an unreliable network, concurrency-control, and mutation.

Our work. This paper is about proving SEC and functional correctness of op-based CRDTs. To the best of our knowledge, all prior work on verification of op-based CRDTs consists of techniques that produce modular specifications but work at the protocol level, or techniques that work for implementations but are non-modular (see Section 7 for a classification of prior work). The main contribution of our work is to lift that restriction: we can produce *modular* specifications of CRDT *implementations*. Additionally, unlike prior work which assumes causal delivery by the network, our CRDTs include a general-purpose implementation of reliable causal broadcast. All our proofs are mechanized in Coq. More precisely, the contributions of this work are as follows:

[3]Delivery occurs when the event processing layer makes its clients aware of the event; this can take different forms depending on the specific application.

[4]The terminology is not universal: Shapiro et al. [2011a] refers to both properties together as *eventual convergence*.

(1) We implemented and verified an RcbLib library for reliable causal broadcast (RCB). To the best our knowledge, this is the first time a formalization of op-based CRDTs includes a general-purpose implementation of RCB, as opposed to assuming causal broadcast.
(2) On top of the RcbLib library, we implemented and verified an OpLib library for building op-based CRDTs. Using OpLib, one can create op-based CRDTs as purely-functional data structures, without having to reason about low-level details like mutation, concurrency control, and network operations. Similarly, by proving only simple sequential specifications, OpLib users obtain from the library rich specifications for their CRDTs, enabling modular reasoning about convergence, causality, and functional correctness.
(3) We evaluated OpLib by implementing a collection of 12 CRDTs, including multiple versions of registers and sets, as well as two combinators for products and maps. We further evaluated the modularity of our specifications by verifying a client program that uses a CRDT obtained via OpLib.
(4) We wrote our libraries in a subset of OCaml that is then automatically translated to AnerisLang, the programming language of the Aneris [Krogh-Jespersen et al. 2020] distributed separation logic. Our proofs were conducted in Aneris and are mechanized in Coq.

Structure of the paper. The rest of the paper is organized as follows: Section 2 gives a quick primer to the Iris and Aneris program logics. Section 3 provides an overview of the key ideas of our work and presents the concepts that CRDT implementers need to use our libraries. Section 4 describes in more detail RcbLib's implementation and correctness proof. Section 5 then does the same for OpLib. Section 6 discusses our case studies (the implemented CRDTs). We then take a look at prior work on Section 7, and conclude in Section 8.

2 ANERIS PRIMER

Iris [Jung et al. 2018] is a state-of-the-art program logic designed to reason about concurrent programs based on separation logic. Aneris [Krogh-Jespersen et al. 2020] is a program logic built on top of Iris for reasoning about distributed systems. Figure 1 shows the fragment of Iris and Aneris logic that we need in this paper:

$$
\begin{aligned}
P, Q \in \mathit{iProp} ::= {} & \mathsf{True} \mid \mathsf{False} \mid P \wedge Q \mid P \Rrightarrow Q \mid P \vee Q \mid \forall x.\, P \mid \exists x.\, P \mid \cdots && \text{higher-order logic} \\
& \mid P * Q \mid P \mathrel{-\!\!*} Q \mid \ell \mapsto_{ip} v \mid \{P\}\ \langle ip; e\rangle\ \{x.\, Q\} \mid \Box P && \text{separation logic} \\
& \mid \boxed{P}^{\mathcal{N}} \mid {}^{\mathcal{E}_1}\!\Rrightarrow^{\mathcal{E}_2} && \text{Iris resources and invariants}
\end{aligned}
$$

Fig. 1. The fragment of Iris and Aneris relevant to this paper

First and foremost Iris is a higher-order logic with the usual connectives. Note how we can quantify, both existentially and universally, over any domain, including *iProp* itself (we write *iProp* for the universe of Iris propositions). Iris is a separation logic. Iris propositions can assert ownership of resources and express their *disjointness*. The proposition $P * Q$ holds if the owned resources can be split into two disjoint parts where one satisfies P and the other Q. The *magic wand*, $P \mathrel{-\!\!*} Q$, also called separating implication, asserts ownership over resources that when combined with (disjoint) resources satisfying P would satisfy Q. The so-called points-to proposition, $\ell \mapsto_{ip} v$, asserts exclusive ownership over the memory location ℓ stating that the value stored in this location is v. This proposition differs from the standard separation logic points-to proposition only in that it is annotated with the *Ip* address of the node to which it belongs — this is necessary as we are working with a distributed system in Aneris. Similarly, in Aneris a Hoare-triple $\{P\}\ \langle ip; e\rangle\ \{x.\, Q\}$, in addition to the program, also takes the *Ip* address of the node the program is running on.

The persistently modality, $\Box$, captures duplicability of propositions. It allows us to distinguish between propositions that are duplicable and those that are not, *e.g.*, points-to propositions: $\ell \mapsto_{ip} v * \ell \mapsto_{ip} w \vdash \mathsf{False}$. Here, $\vdash$ is the logical entailment relation of Iris. Intuitively, $\Box P$ holds if P does and furthermore, P does not assert ownership of any non-duplicable resources. We say a proposition is persistent if $P \vdash \Box P$; note that for any proposition P we always have $\Box P \vdash P$. Persistent propositions are duplicable, *i.e.*, $\Box P \vdash \Box P * \Box P$, and hence they merely express knowledge as opposed to expressing (exclusive) ownership over resources. An example of a persistent proposition is Iris invariants. The invariant $\boxed{P}^{\mathcal{N}}$ asserts that P must hold at all times throughout program execution. Hence, throughout a proof, for the duration of an atomic step of computation, we can access invariants, *i.e.*, we get to know that the invariant holds before the step of computation and need to guarantee that it also holds afterwards. The name of the invariant $\mathcal{N}$ is used to track accesses to invariants and prevent them from being accessed in an unsound manner, *e.g.*, accessing the same invariant twice during the same atomic step of computation which could result in duplicates of non-duplicable propositions like the points-to proposition. The update modality,[5] ${}^{\mathcal{E}_1}\Rrightarrow^{\mathcal{E}_2}$, allows manipulation of invariants and resources in Iris. The masks $\mathcal{E}_1$ and $\mathcal{E}_2$ are sets of invariant names and respectively indicate which invariants hold before and after the "update" takes place. We write $\Rrightarrow_{\mathcal{E}}$ for ${}^{\mathcal{E}}\Rrightarrow^{\mathcal{E}}$. The update modality is the primary way of working with invariants in Iris. They are used in the definition of Iris Hoare-triples in such a way as to enforce the aforementioned invariant policy of only allowing access to invariants during atomic steps of computation. Intuitively, the proposition ${}^{\mathcal{E}_1}\Rrightarrow^{\mathcal{E}_2} P$ holds if we can manipulate resources (allocate new resources, or update the existing ones) and manipulate invariants (create new invariants, access invariants, or reestablish invariants) so as to make sure that P holds. Furthermore, during this update we can access all invariants in $\mathcal{E}_1$ but must ensure that all invariants in $\mathcal{E}_2$ hold after the update is done.

3 MAIN IDEAS

This section provides a birds-eye view of the paper, focusing on concepts users need to use our libraries. Figure 2 shows an overview of our work. We structured our development as a tower of components, each exporting a modular specification.

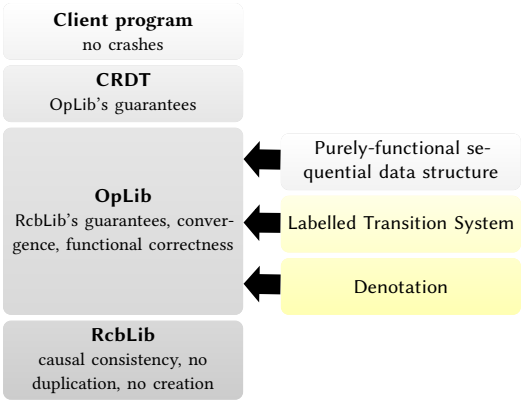


Fig. 2. Overview of our development. The OpLib library is parametrized by a CRDT specification given by the components in the right column. Grey boxes are written in OCaml/AnerisLang; yellow boxes are written in Coq.

[5] In Iris jargon this modality is called the fancy update modality; see Jung et al. [2018] for more details.

Higher-level components can then be verified using solely the specifications of its dependencies, without knowledge of the dependency's implementation. Each box in Figure 2 lists a component and the safety properties guaranteed by its specification. Grey boxes are written in OCaml; [6] yellow boxes are written in Coq.

3.1 RcbLib

At the base of our verified tower of components we have a library implementing a reliable causal broadcast protocol [Cachin et al. 2011]. This library is built on top of UDP, so it makes minimal assumptions about network guarantees. In particular, messages can be dropped, re-ordered, and duplicated by the network. The library deploys a suite of techniques, such as sequence ids, acknowledgments, retransmissions, and a delay queue, to offer three main guarantees: broadcast messages are delivered in causal order, without duplicates, and ensuring that any message delivered was previously broadcast by another participant (the *no creation* property in Figure 2). These are the three safety properties of RCB [Cachin et al. 2011].

Verifying RcbLib. The main idea for verifying RcbLib is to generalize the treatment of causality in Gondelman et al. [2021] to the causal broadcast setting. We now briefly outline our approach and expand on it in Section 4.

The first step is to define separation logic resources tracking the set of broadcast messages between replicas in two ways: the $\mathsf{OwnGlobal}(h)$ resource provides a *global view* tracking the set h of all messages broadcast by any replica, while the $\mathsf{OwnLocal}(i, s)$ resource provides a *local view* tracking the set s of all messages that has been delivered by replica i. Here, messages are triples $(\mathsf{p}, \mathsf{vc}, \mathsf{o})$ consisting of the message's payload p, vector clock vc, and id of the originating replica o.

The next step is to craft separation logic specifications for RcbLib's broadcast and deliver functions. Below, we show a simplified specification for broadcast :

$$\{\mathsf{OwnGlobal}(h) * \mathsf{OwnLocal}(i, s)\}$$
$$\langle ip_i; \mathsf{broadcast}(p)\rangle$$
$$\{m.\, \mathsf{payload}(m) = p * \mathsf{OwnGlobal}(h \uplus \{m\}) * \mathsf{OwnLocal}(i, s \uplus \{m\})\}$$

This spec states that in order to broadcast a message with payload p, we need to provide both the global view and the local view of the broadcasting replica. broadcast can then execute without errors and return a message m with payload p. Logically, we know that the global set of broadcast messages now includes m, and also that node i has delivered (is aware of) the new message.

In addition to the broadcast and deliver specifications, following Gondelman et al. [2021] we provide to the user of RcbLib a set of laws governing the above resources. Notably, the causality law states that, given the ownership of $\mathsf{OwnGlobal}(h)$ and $\mathsf{OwnLocal}(i, s)$, we can conclude that

$$\forall m \in s, m' \in h.\ \mathsf{vc}(m') < \mathsf{vc}(m) \Rightarrow m' \in s$$

i.e., for any message m that has been delivered at node i, if we know of another message m' that has been broadcast by any other node such that m' happened before m, [7] then it must be the case that node i has previously delivered m' as well. All laws are proven in Coq and provided as lemmas.

3.2 OpLib

Conceptually, an op-based CRDT implementation can be seen as an infinite loop that maintains the CRDT's state at a given replica. This loop has a number of responsibilities:

(1) accept *local* operations invoked by the user at the replica

[6] Later automatically translated to AnerisLang, the programming language of the Aneris distributed separation logic.
[7] $\mathsf{vc}(m)$ stands for m's *vector clock*, a mechanism for tracking causal dependencies.

(2) modify the CRDT's state as per the effects of local operations
(3) propagate local operations to other replicas
(4) listen for *remote* operations communicated via the network
(5) modify the CRDT's state as per the effects of remote operations

One can then observe that there are a number of derived responsibilities that flow from the ones above: for example, since steps (2) and (5) can happen concurrently, some form of concurrency control (e.g. locking) is needed. Additionally, because the network is unreliable, step (3) requires that the CRDT is be able to tolerate dropped messages. Another observation is that most of the steps above are agnostic to the semantics of the specific CRDT: only when modifying the CRDT's state (steps (2) and (5)) do we need to know the inner workings of the data type's operations.

These observations suggest a design where the generic responsibilities are factored out as a library that is parametric on the CRDT's operations and their effects. Inspired by the approach in Baquero et al. [2014], we instantiate a CRDT via the OpLib library that we have implemented on top of RcbLib. In our library, all that the user needs to provide is the data type's *initial state* and an *effect* function that can process new operations. This design allows a CRDT implementer to focus on the core logic of their data type as a purely-functional data structure, while delegating to OpLib all the gritty details of inter-replica communication, concurrency control, and mutation. Because OpLib uses RcbLib for propagating operations between replicas, clients can rely on the guarantees of causal broadcast. Once instantiated with the user's purely functional data type, OpLib turns it into a fully-fledged CRDT that exports two functions: get_state(), which returns (a copy of) the CRDT's current state, and update(op) which updates the state via a new operation *op*.

Verifying OpLib. To verify OpLib we adapt the notion of CRDT *denotations* [Burckhardt et al. 2014; Leijnse et al. 2019] to separation logic. A CRDT denotation $[\![\cdot]\!] : 2^{Msg} \rightharpoonup St$ is a (partial) function from sets of messages (a message contains an operation plus causality metadata) to the CRDT state that results from executing said operations. Both *Msg* and *St* vary depending on the specific CRDT. For example, the denotation for a PN-Counter is a function that maps a set of messages to the sum of its payloads: $[\![s]\!] = \sum_{m \in s} \mathsf{payload}(m)$.

Denotations have been previously used to give high-level specifications for CRDTs as well as CRDT combinators (e.g. products of CRDTs and maps where the value type is an arbitrary CRDT) [Burckhardt et al. 2014; Leijnse et al. 2019]. However, those works do not use denotations to verify implementations. We adapt denotations by constructing a separation logic resource $\mathsf{LocSt}(i, \bullet s, \circ r)$[8] which tracks the sets s and r of local and remote operations, respectively, processed by replica i. The key insight behind the resource $\mathsf{LocSt}(i, \bullet s, \circ r)$ is that it tracks *precisely* the set of processed local operations s, but provides only *a lower bound* on the set of processed remote operations r. This captures the intuition that while a CRDT user can control which local operations they perform, they do not know which additional remote operations have been propagated from other replicas at a given moment in time. The simplified spec for get_state below shows how the resource is used:

$$\{\mathsf{LocSt}(i, \bullet s, \circ r)\}\ \mathsf{get_state}()\ \{m.\ \exists r', r \subseteq r' * m = [\![s \cup r']\!] * \mathsf{LocSt}(i, \bullet s, \circ r')\}$$

The spec says that prior to calling get_state we must know that replica i has processed exactly the local messages in s, and at least the remote messages in r. The function then returns a state m that is the denotation of the set $s \cup r'$, where r' is a superset of r. This is because in between calls to get_state the CRDT might have processed additional remote operations.

3.3 CRDT Instances

The last element of Figure 2 we highlight is the recipe that CRDT implementer follow to use OpLib:

[8]The notation is reminiscent of the so-called authoritative resource algebra [Jung et al. 2018].

- First, the CRDT implementer must provide a denotation for their CRDT.
- In order to bridge the abstraction gap between the denotation, stated in terms of the sets of operations, and the effect function, which must process one operation at a time, the user provides a second specification in the form of a *labelled-transition system* (LTS). In this LTS, states are the CRDT's states and the transitions are labelled with operations. That is, a transition $s \xrightarrow{op} s'$ means that if the CRDT is in state s and an operation op is received, then it will end up in state s'. Importantly, the denotation and LTS must agree in the following sense: if h is a set of operations such that $[\![h]\!] = s$, and $s \xrightarrow{op} s'$, then we must have $[\![h \cup \{op\}]\!] = s'$.
- Finally, the user shows that their effect function is coherent with the LTS via a Hoare triple.

The first two steps are conducted outside separation logic in the meta-logic (Coq), while the last step requires proving a Hoare triple in Aneris.

We have followed the recipe above to implement 12 CRDTs, including multiple kinds of registers and sets, as well as two CRDT combinators for products and maps. Our combinators use Coq typeclasses as in Liu et al. [2020] to automatically generate and prove correctness of compound CRDTs from constituent CRDTs.

Our examples come from the CRDT literature [Baquero et al. 2014; Leijnse et al. 2019; Shapiro et al. 2011a]. Importantly, they include CRDTs where all operations naturally commute (e.g. PN-Counter) as well as others that require causality information to make operations commutative (e.g. Last-Writer-Wins Register and Add-Wins Set). This shows that our approach scales to different CRDT designs.

4 RELIABLE CAUSAL BROADCAST

The network primitives (send and receive) provided by AnerisLang are for *point-to-point* communication: that is, a process communicating with a single other process. They are also, as previously mentioned, unreliable in a number of ways: messages can get lost, duplicated, and re-ordered in transit.

A useful abstraction in distributed systems is that of *broadcast*. In broadcast, or *one-to-many* communication, a process transmits the same message to one or more other processes. There exist different broadcast algorithms providing different guarantees: one such kind is *reliable causal broadcast* (RCB). In RCB, clients are provided with two operations, broadcast(msg) and deliver() that satisfy the following properties (taken from Cachin et al. [2011] and classified as either liveness or safety properties):

- (RCB1, liveness) *Validity*: if a correct process p broadcasts a message m, then p eventually delivers m.
- (RCB2, **safety**) *No duplication*: no message is delivered more than once.
- (RCB3, **safety**) *No creation*: if a process delivers a message m with sender s, then m was previously broadcast by process s.
- (RCB4, liveness) *Agreement*: if a message m is delivered by some correct process, then m is eventually delivered by every correct process.
- (RCB5, **safety**) *Causal delivery*: for any message m_1 that potentially caused a message m_2, i.e., $m_1 \rightarrow m_2$, no process delivers m_2 unless it has already delivered m_1.

In this section, we sketch our implementation of a library for RCB, RcbLib, based on Birman et al. [1991] and Baquero et al. [2014]. We proved specifications of our implementation that satisfy the three safety properties above. In fact, our RCB library implements a slightly stronger specification than regular RCB, because it exposes to its clients causality information associated to messages in

the form of vector clocks. The additional information provided by this *tagged* form of RCB [Baquero et al. 2014] simplifies the task of building CRDTs using OpLib (see Section 6).

4.1 Implementation

Since AnerisLang's network primitives provide few guarantees, RcbLib deploys a few different techniques in order to achieve the safety properties just mentioned. Some of the challenges and their solutions are outlined in Table 1. Additionally, Figure 3 provides a high-level view of the design of the RCB algorithm. The main components are outlined below.

Table 1. Challenges and techniques employed in RCB's implementation

Challenge	Technique
Messages can be dropped, reordered and duplicated by the network.	Stop-and-wait protocol [Tanenbaum and van Steen 2007] using sequence ids, acknowledgments, and retransmissions to handle unreliable network.
The broadcasting process can be partitioned from the network before all processes receive a broadcast.	Eager reliable broadcast (retransmissions) [Cachin et al. 2011].
Messages need to be delivered in causal order.	Delay delivery of messages until causal dependencies are satisfied, using a *delay queue* and *vector clocks* [Birman et al. 1991].

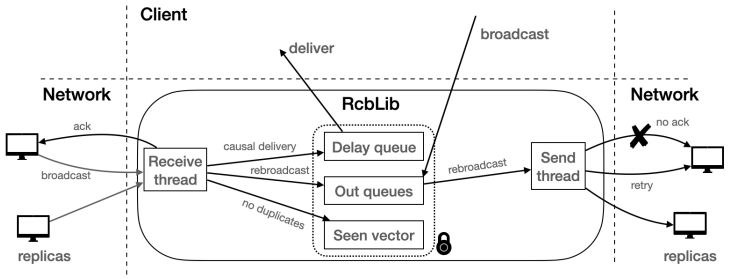


Fig. 3. Structure of the reliable causal broadcast library

Receive and send threads. RcbLib consists of two concurrent threads that operate on a set of shared data structures (concurrent accesses are synchronized via a lock). The *receive* thread listens for messages on a network socket and places them in a *delay queue* and a collection of *out-queues*. It also acknowledges received messages so other replicas can move on to broadcasting new messages.

The *send thread* sends the messages in the out-queues to other replicas following a *stop-and-wait* protocol [Tanenbaum and van Steen 2007]. That is, a message is repeatedly sent to another replica until it is acknowledged by the foreign replica; at which point the send thread pops the relevant out-queue and moves on to a not-yet-acknowledged message.

Library API. The library has two client APIs: deliver and broadcast. The former removes a message m from the delay queue such that all of the message's causal dependencies have previously been delivered (i.e. a message that comes next according to causal order). If no such message exists, deliver returns None; otherwise it returns Some(m). The broadcast function broadcasts a message to all replicas (except to the current one). It does so by placing the message in all out-queues, so it can be later picked up by the send thread. broadcast(p) returns a new message m' containing

the payload p together with the vector clock assigned to m' and the issuing replica's id. Because a replica doesn't broadcast to itself it must use the return value of broadcast if it wants to process the newly-broadcast message m'.

Vector clocks. We use vector clocks to keep track of logical time [Fidge 1987; Mattern et al. 1988]. A vector clock is an array of non-negative integers; there is one array entry per replica in the system, and each entry records the number of events that originate at the corresponding replica. It is possible to merge two vector clocks by taking the maximum of their entries pointwise. We can define a partial order $\leq_{vc}$ on vector clocks by lifting $\leq$ (from $\mathbb{N}$) pointwise. The following result then holds: let a and b be events. then $a \rightarrow b$ iff $vc(a) < vc(b)$.

Replicas maintain internal state with their current vector clock. Every sent message m is also tagged with a vector clock $vc(m)$. When broadcast is called, the replica increments its entry within the internal vector clock and tags the event with it. When the receive thread receives a new message, its vector clock is not immediately merged with the replica's vector clock; instead, the merge is delayed while the message waits in the delay queue.

Delay queue. In order to ensure causal delivery of messages, RCB stores messages received from other processes in a delay queue. That is, we do not deliver received messages immediately to the user. Given the internal vector clock v_i and a message m from the delay queue, we can determine whether (a) all causal dependencies of the message have been previously delivered and (b) the message has not been previously delivered. We do this using the following *delivery condition* [Birman et al. 1991]:

$$\text{canDeliver}(m, v_i) \triangleq \forall k \in \{1 \ldots n\} \begin{cases} vc(m)[k] = v_i[k] + 1 & \text{if } k = origin(m) \\ vc(m)[k] \leq v_i[k] & \text{otherwise} \end{cases}$$

Once the delivery condition for m is met, it is safe (causally consistent) to deliver m to the user in the next invocation of deliver. At that point, the internal vector clock v_i can be updated by merging it with $vc(m)$.

Out queues. Consider the following scenario. There are three processes A, B and C. A broadcasts a message m to B and C. After A has sent m to B, but before it has a chance to send it to C, the network becomes partitioned into two partitions $\{A\}$ and $\{B, C\}$. Now B receives m, but C will not receive m until the partition is healed. This violates the agreement (RCB4) property of Section 4 because the partition might never heal, so C might never get m. Additionally, suppose that B creates a new message m', which is now causally dependent on m: $m \rightarrow m'$. Even though B and C are in the same partition, C cannot deliver m' until it delivers m first (a causal dependency). The whole system is stuck because one process is partitioned.

For this reason, RCB implements a form of *eager reliable broadcast* [Cachin et al. 2011]. That is, every process re-broadcasts every single message received to every other process (taking care to not enter into loops). Eager rebroadcasting is inefficient, since for every message sent there are $O(n^2)$ re-broadcasts in a system with n replicas (as opposed to $O(n)$, which is the best case for broadcast). We have chosen this mechanism for the first iteration of the RCB library due to its simplicity.

Given the need to re-broadcast messages, and because the network is unreliable, each process maintains a set of *out queues*, one per other process in the system (so n queues per node). Each queue contains the outbound messages that need to be sent to a specific process, but have not yet been acknowledged by that process. Messages are copied from the delay queue to the out queues, and are removed from the out queues when acknowledged by the intended recipient.

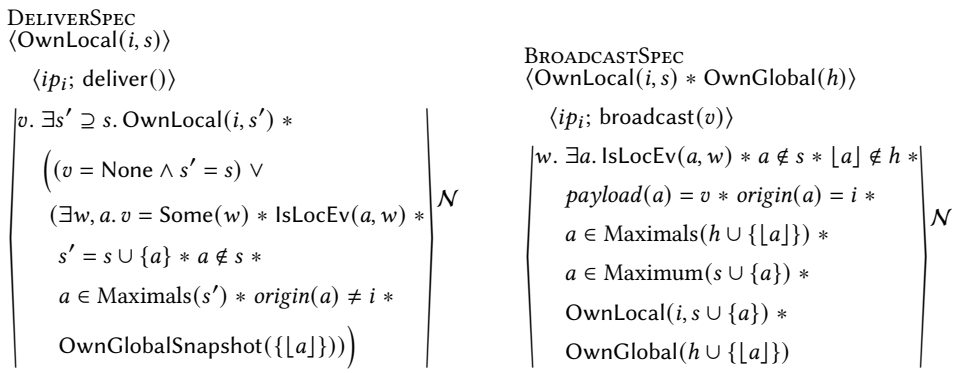


Fig. 4. Logically-atomic specifications for deliver and broadcast. $\mathcal{N}$ is any namespace containing the global invariant's name.

Seen vector. A message could be received multiple times by the same process: because the network generated a duplicate or the message was re-broadcast multiple times by other processes. In either case, we need a mechanism to avoid re-delivery of the same message; in other words, we need to avoid putting the same message twice in the delay queue. To this effect, we use vector clocks as sequence identifiers. Given a message m, the pair $(origin(m), vc(m)[origin(m)])$ uniquely identifies a message in the system. We can then construct a *seen vector* where the ith entry gives us the highest sequence id of a message originating from process i that has been previously received. We only place a message originating at process i in the delay queue if its sequence id is higher (by one) than the current value of seen$[i]$.

4.2 Specification

As mentioned in Section 3, the specifications for deliver and broadcast (shown in Figure 4) use separation logic resources that keep track of the local and global states of the broadcast. The local resource OwnLocal(i, s) tells us that in process i RcbLib has previously delivered exactly the messages in s. Similarly, the global resource OwnGlobal(h) implies that h is exactly the set of messages that have been broadcast by any replica. We also maintain a *global invariant* RcbInv that ensures that global and local states are compatible. The invariant states that at all times if we combine all local states we obtain the global state, and furthermore that the local states satisfy causal delivery.

Deliver. Figure 4 shows the specification of RCB's deliver function. The intuition is that before calling deliver we should know which messages have been previously delivered at this process (via ownership of a resource OwnLocal(i, s)). After deliver returns, there are two possibilities:

- No messages were available for delivery, so the function returns None, and we get back our unchanged OwnLocal(i, s).
- There was a message a available for delivery. In this case, the function returns Some(w), where w is the physical counterpart to a, reflected by the predicate IsLocEv(a, w). Additionally, we receive back a resource OwnLocal$(i, s \cup \{a\})$. That is, we logically record the delivery of the new message. Crucially, we know that $a \notin s$, meaning that the returned message has not been previously delivered. Additionally, we get to know that a is *maximal* with respect to vector clock order in the set $s \cup \{a\}$. This means that no previously-received message could causally depend on a (but a can depend on previous messages). Finally, we obtain the

resource OwnGlobalSnapshot($\{\lfloor a \rfloor\}$), [9] which serves as proof that the returned message $\lfloor a \rfloor$ did not "come out of thin air": it was properly recorded in the global state. In general, owning a *global snapshot* OwnGlobalSnapshot(r) gives us a lower bound r on the set of all messages sent: if we own both OwnGlobal(h) and OwnGlobalSnapshot(r) we can conclude $r \subseteq h$. The OwnGlobalSnapshot(r) resource is persistent (Section 2), meaning that we can make copies of it freely; this makes snapshots useful as certificates that a certain message was broadcast by RCB.

Broadcast. Figure 4 also shows the specification of broadcast. Intuitively, the effect of broadcast is to generate a new message, which in our framework needs to be recorded both as part of the global state as well as of the local state of the process calling broadcast. This is why in the precondition of broadcast we need to provide both OwnGlobal(h) and OwnLocal(i, s). The function then returns a local event w and its logical representation a, as evidenced by the predicate IsLocEv(w, a). A few points worth pointing out:

- Unlike in traditional implementations of RCB, where broadcast returns unit, our broadcast returns the generated message (or local event) corresponding to the broadcast value. For example, if replica i broadcasts the value 2, then broadcast(2) returns a tuple $(2, \text{vc}, i)$ for some vector clock vc that is globally maximal. In general, the return value is of the form (payload, vc, origin). This is why we call our implementation *tagged* RCB, as per Baquero et al. [2014].
- As expected, the newly generated message has not been previously recorded. This is given by $a \notin s$ and $\lfloor a \rfloor \notin h$.
- We obtain back resources OwnGlobal($h \cup \{\lfloor a \rfloor\}$) and OwnLocal($i, s \cup \{a\}$), showing that the event has been properly recorded both locally and globally.

Logical Atomicity. The observant reader might have noticed two peculiar points about the specs above.

First, the broadcast spec requires the user to provide the global state resource OwnGlobal(h). Separation logic is all about modular specification, so a global resource that tracks all broadcast events would seem to be antithetical to separation logic. However, we find that the global resource is useful when reasoning about closed programs, because it allows us to state invariants of the form "all messages ever sent satisfy a safety property P" (e.g. in a system with two replicas, all messages are sent by one of the replicas).

A more practical concern is how to get two processes to concurrently broadcast messages, since it would seem that the broadcast spec requires exclusive ownership of OwnGlobal(h); it in fact does not. The reason is that our specs do not use regular Hoare triples, but instead rely on *logically-atomic triples* [Jung et al. 2015]. Instead of the regular $\{P\}\ e\ \{Q\}$ we write $\langle P \rangle e \langle Q \rangle_{\mathcal{N}}$. The intuition is the following: if we can prove the atomic triple above, then e is evaluated until a certain step (its *linearization point* [Herlihy and Wing 1990]) at which point P holds, possibly after opening any invariant that is not in the $\mathcal{N}$ namespace. After the atomic step, Q then holds, and all opened invariants need to be closed. So Q does not necessarily hold when the function terminates, but it always holds after the linearization point. The advantage of atomic triples is that we are allowed to open invariants when proving the precondition P. This is useful in the broadcast spec, because the global resource OwnGlobal(h) is likely to be kept in an Iris invariant by most clients of RCB

[9] The notation $\lfloor a \rfloor$ stands for the *erasure* of a. This is a technical detail we inherited from the development in Gondelman et al. [2021], because we represent local and global events differently. The erasure of a local event a gives us the corresponding global event $\lfloor a \rfloor$.

(otherwise clients will not be able to concurrently broadcast messages). Our definition of atomic triples is adapted from that in Perennial [Chajed et al. 2019].

Resource lemmas. As mentioned in Section 3.1, in addition to the specs above and the resources that track messages, we proved a number of lemmas (e.g. causality) that serve as reasoning principles for using the resources. Because our treatment of causality is an adaptation of Gondelman et al. [2021], the reader can consult that paper for the full list of resource lemmas.

Safety Properties. We now show how RcbLib satisfies the three the safety properties presented in Section 4.

(RCB2) No duplication. This property follows from the deliver spec (Figure 4); specifically, the postcondition guarantees that the delivered message a (if one exists), was not previously delivered to the same process ($\mathsf{OwnLocal}(i, s \cup \{a\}) * a \notin s$).

(RCB3) No creation. We prove this as a property of local state resources:

$$\boxed{\mathsf{GlobalInv}}^{\mathcal{N}_{\mathsf{GI}}} * \mathsf{OwnLocal}(i, s_i) * \mathsf{OwnLocal}(j, s_j) * e \in s_i * \mathit{origin}(e) = j \vdash \\ \Rightarrow\!\!\!\!\Rightarrow_{\mathcal{E}} \exists e'.e' \in s_j \wedge \lfloor e' \rfloor = \lfloor e \rfloor$$

Here, you can imagine i as the process that has just received message e. If i can assert that m originated in process j, and we also have knowledge of the local state of j in the form of $\mathsf{OwnLocal}(j, s_j)$, then the lemma guarantees that e is in fact also present in s_j (or, more precisely, that one can find messages in both local histories with equal erasures). The lemma above holds in the presence of a *global invariant* $\boxed{\mathsf{GlobalInv}}^{\mathcal{N}_{\mathsf{GI}}}$ that RcbLib maintains to coordinate the local state resources of different replicas.

(RCB5) Causal delivery. This is the main resource lemma, which was already informally described in Section 3.1. The full form also holds under the global invariant, and uses global snapshots instead of the full global state:

$$\boxed{\mathsf{GlobalInv}}^{\mathcal{N}_{\mathsf{GI}}} * \mathsf{OwnLocal}(i, s) * \mathsf{OwnGlobalSnapshot}(h) \vdash \Rightarrow\!\!\!\!\Rightarrow_{\mathcal{E}} \forall a \in s, w \in h.\ vc(w) < vc(a) \Rightarrow \\ \exists a' \in s.\ \lfloor a' \rfloor = w$$

4.3 Correctness Proof and Its Relationship to Gondelman et al. [2021]

As we mentioned in Section 3.1, our proof that RcbLib's implementation meets the specifications in Figure 4, as well as our proofs of the safety lemmas that follow under the global invariant, are based on the proof recipe outlined in Gondelman et al. [2021]. Gondelman et al. [2021] implement and specify a causally-consistent distributed key-value store, also within separation logic using Aneris. The proof recipe they outline (which we follow) can be summarized thus:

- First, model the distributed system as a state-transition system, where each state tracks the set of events at each replica. [10] Additionally, we track the global state of the system as the union of local events.
- Next, we embed the model in separation logic by using Aneris's ghost theory to create separation logic resources that represent knowledge of the local and global states. For example, Gondelman et al. [2021] construct a resource $\mathsf{Seen}(i, s)$ indicating that replica i has received *at least* the writes in s. Our analogous resource is $\mathsf{OwnLocal}(i, s)$, which captures the knowledge the replica i has delivered *exactly* the messages in s.

[10] For them, an event is a write to the key-value store; for us, an event is a delivered message.

$$\Sigma = \mathbb{N} \qquad \sigma_i^0 = 0 \qquad \mathsf{prepare}_i(\mathsf{inc}, n) = \mathsf{inc} \qquad \mathsf{effect}_i(\mathsf{inc}, n) = n + 1 \qquad \mathsf{eval}_i(\mathsf{rd}, n) = n$$

Fig. 5. Specification of op-based counter CRDT from Baquero et al. [2014]

- Construct a global invariant (another proposition) that implies that the aforementioned resources describe reachable states in the state-transition system. For example, if we own $\mathsf{OwnLocal}(i, s)$, we can then conclude (provided the global invariant holds) that s is not an arbitrary set of messages, but instead satisfies certain safety properties (e.g. s is causally-closed, the origin field of messages is in the right range, etc.). This is also the step where we prove the resource laws (e.g. causality and no-creation).
- Finally, to verify the code running in each replica, establish a *lock invariant* [Birkedal and Bizjak 2017] that tracks the set of events that have been processed by the replica so far. In doing so, one has to carefully pick the right (combination of) resource algebras (RAs) from which to draw the separation logic resources, so that the right properties hold and invariants can be preserved.

We were also able to reuse part of Gondelman et al. [2021] Coq's development in our proof of RcbLib. To be clear, we do not claim the proof recipe above as our contribution. Instead, our contribution is producing for the first time modular specifications for a general-purpose library for causal broadcast. By contrast, Gondelman et al. [2021] deal with causality specifically within the context of a key-value store. In addition, our implementation includes multiple techniques to improve reliability (e.g. sequence ids, acknowledgements, eager re-broadcasts) that are not present in Gondelman et al. [2021]. See Section 7 for additional details.

5 A LIBRARY FOR IMPLEMENTING CRDTS

Figure 5 shows a specification for a counter CRDT[11] taken from Baquero et al. [2014]. This is not a separation-logic specification; instead, the counter is specified by instantiating several generic components: a set of states Σ (the naturals), an initial state (0), and a function effect that given a counter state and an operation returns the resulting state (the counter has only one kind of operation: add). [12] This style of specification is used throughout the CRDT literature [Baquero et al. 2014; Burckhardt et al. 2014; Shapiro et al. 2011a] and it is a useful one because it allows us to focus on the parts of a CRDT that are truly unique to the CRDT in question. By contrast, the spec leaves many details unspecified: how are messages sent from one replica to others (some kind of broadcast), what happens when the current replica tries to update its state concurrently with a remote update being processed (we need locking), how is the replica state persisted across operations (mutable state). These details are common across different CRDTs, so it would be useful to factor their implementation into a separate library that can then be instantiated by CRDT implementer. This is what we have done with our OpLib library, which reuses our RCB implementation from Section 4 to provide the scaffolding for implementing op-based CRDTs.

5.1 Implementation

OpLib's code is shown in Figure 6. To use the library, the user calls oplib_init and provides serialization and deserialization functions (ser and deser) for the CRDT's operations, together with

[11] Sometimes referred to as a *grow-only* or G-Counter, because the counter can only be incremented.
[12] The spec also shows two other functions: prepare which builds an "internal" operation from an "external", user-provided operation (this can often by taken to be just the identity); and eval which queries the CRDT's state.

```
let oplib_init ser dser addrs rid crdt =
  let res = rcb_init ser dser addrs rid in
  let (del, br) = res in
  let crdt_res = crdt () in
  let (init_st, eff) = crdt_res in
  let st = ref (init_st ()) in
  let lock = newlock () in
  fork (apply_thread lock del st) eff;
  (get_state lock st, update lock br st eff)

let get_state lock st () =
  acquire lock;
  let res = !st in
  release lock;
  res
```

```
let update lock br st effect op =
  acquire lock;
  let msg = br op in
  st := effect msg !st;
  release lock

let apply_thread lock del st eff =
  loop_forever (fun () ->
      acquire lock;
      begin
        match (del ()) with
          Some msg -> st := eff msg !st
        | None -> ()
      end;
      release lock;)
```

Fig. 6. Code of OpLib library

the addresses of replicas (addrs), the current replica id (rid) and most importantly the logic for the specific CRDT being implemented (crdt). The crdt value has the following polymorphic type:

```
type repIdTy = int  (* replica id *)
type 'opTy msgTy = ('opTy * vector_clock) * repIdTy
type ('opTy, 'stateTy) effectFnTy = 'opTy msgTy -> 'stateTy -> 'stateTy
type ('opTy, 'stateTy) crdtTy = 'stateTy * ('opTy, 'stateTy) effectFnTy (* init st, effect *)
```

That is, as in Figure 5, a CRDT is specified by its initial state and an effect function that knows how to transition from a state to the next. Unlike Figure 5, however, we now have executable OCaml code instead of a high-level specification.

Going back to oplib_init, the function uses the RcbLib library to obtain a pair of functions for delivering (receiving) and broadcasting messages to other replicas. It then allocates a reference to store the CRDT state (starting with the initial state provided by the user) and then forks an apply_thread that listens for messages sent by remote replicas, so we can apply their effects. Finally, oplib_init returns a pair of functions (get_state, update) that the user can call to query the CRDT's state and update it, respectively.

The apply_thread function runs an infinite loop that first tries to deliver the next message in causal order (using RcbLib) and then, if one exists, updates the CRDT's state using the user-provided effect function.

Finally we have the user-facing functions get_state and update. The former returns a copy of CRDT's current state; the latter uses RCB to broadcast the new operation op to other replicas. RcbLib returns the user-provided operation wrapped with causality information (so an operation becomes a message); update then uses the newly-created message and the effect function to update the CRDT state.

Three points of note: first, effect is a pure function: given a state and a message it returns the resulting state. Second, concurrent accesses to the internal state (e.g. concurrent executions of apply_thread and update) are synchronized via a lock. Finally, notice that OpLib does not directly invoke any network operations (e.g. creating a network socket, sending a message, etc.); instead, all of the networking functionality is encapsulated in RcbLib.

5.2 Specification

We start by arguing why, for CRDTs, resources like $\mathsf{LocSt}(i, \bullet s, \circ h)$ that track the set of executed operations are preferable to those that track the CRDT's state. Another way to say this is that CRDTs benefit from having *intensional*[13] specifications.

From counters to replicated counters. Consider a simple counter module exposing two functions: incr() increases the counter's value by one, and read() returns the counter's current value. If used in a sequential setting, one can imagine being able to prove the following specifications: $\{\mathsf{c} \mapsto \mathsf{n}\}\ \mathit{incr}(c)\ \{c \mapsto n+1\}$ and $\{c \mapsto n\}\ \mathsf{query}()\ \{v.v = n\}$. Now we move to a concurrent or distributed setting, where the previous specs are still provable but no longer useful, because we need to be able to increment the counter concurrently. To solve this problem, we can track a lower bound of the counter's value, instead of the counter's exact value. Then every time we increment, we can increment the lower bound by one. This is precisely how Timany et al. [2021] structure their specification of a G-Counter CRDT: they have a resource $\mathsf{gcounter}(i, m)$, meaning that at replica i the counter's value is *at least* m (Figure 7).

$$\{\mathsf{gcounter}(i,k)\}\ \langle ip_i;\ \mathtt{query}()\rangle\ \{m.\ k \le m * \mathsf{gcounter}(i,m)\} \qquad \textsc{QuerySpec}$$
$$\{\mathsf{gcounter}(i,k)\}\ \langle ip_i;\ \mathtt{incr}()\rangle\ \{().\ \exists m.\ k < m * \mathsf{gcounter}(i,m)\} \qquad \textsc{IncrSpec}$$

Fig. 7. G-Counter specification from Timany et al. [2021]

This works but has at least two drawbacks. First, the incr spec is unable to distinguish between a properly-implemented counter and one that increments the state by two instead of one every time incr is called. Second, even if we are able to fix the previous issue, perhaps by tracking "contributions" as in Birkedal and Bizjak [2017], we face an even thornier problem if we consider not an increment-only counter, but one that additionally has a decrement operation. The problem there is what to write in incr's post-condition. Since the counter's state is no longer monotonic, if we start with a $\mathsf{gcounter}(i, m)$, we can end up with a $\mathsf{gcounter}(i, k)$ where k can be greater, equal, or less than m. We have lost all knowledge about the counter's state.

Consider what happens if instead of trying to track the counter's *state* we track the *operations* that the counter has processed. First, it makes sense to split said operations into those that are generated locally and the ones that come from other replicas. This is because a replica "knows" the operations *it* has performed, but it does not know what operations have been performed remotely until query or incr are called. Figure 8 show these new intensional specs. Ownership of the resource $\mathsf{gcounter}(i, s, h)$ conveys knowledge that at replica i we have processed *exactly* the operations in s and *at least* the operations in h. In this case an operation is a pair (inc, i) containing the operation type (we only have one kind of operation: inc) and the replica id. Logically, calling query involves trading our knowledge of $\mathsf{gcounter}(\mathsf{i}, \mathsf{s}, \mathsf{h})$ for knowledge of $\mathsf{gcounter}(i, s, h')$, where $h \subseteq h'$. That is, after calling query we might become aware of additional remote operations, but the set s of local operations does not change. By contrast, in calling incr we trade $\mathsf{gcounter}(i, s, h)$ by $\mathsf{gcounter}(i, s \cup \{(\mathit{inc}, i)\}, h')$ with $h \subseteq h'$. This means that after incr returns the set of local operations has grown by exactly one element (as expected), and also new remote operations might have been processed as well. This specification style solves our problems because it allows us to track what the current thread's contribution is to the counter's state. It also scales well to handling a dec operation: the incr spec would not change; we would just need to adjust query's spec so that

[13] In the sense of Roscoe [1996], as opposed to the more common *extensional* specifications that focus on the observable effects of operations.

$$\{\text{gcounter}(i,s,h)\}\ \langle ip_i;\ \texttt{query}()\rangle\ \{m.\ \exists h' \supseteq h.\ m = |s \cup h'| * \text{gcounter}(i,s,h')\} \qquad \textsc{QuerySpec}$$
$$\{\text{gcounter}(i,s,h)\}\ \langle ip_i;\ \texttt{incr}()\rangle\ \{().\ \exists h' \supseteq h.\ \text{gcounter}(i, s \cup \{(\text{inc}, i)\}, h')\} \qquad \textsc{IncrSpec}$$

Fig. 8. Intensional G-Counter specifications

$$\textsc{GetStateSpec}\quad \langle \text{LocSt}(i, \bullet s, \circ h)\rangle\ \langle ip_i;\ \text{get_state}()\rangle\ \left\langle v.\ \exists h'\ w.\ h' \supseteq h * \text{StCoh}(w,v) * \text{LocSt}(i, \bullet s, \circ h') * [\![s \cup h']\!] = w \right\rangle^{\mathcal{N}}$$

$$\textsc{UpdateSpec}\quad \langle \text{LocSt}(i, \bullet s, \circ r) * \text{GlobSt}(h)\rangle\ \langle ip_i;\ \text{update}(v)\rangle\ \left\langle \begin{array}{l}().\ \exists a\, r'.\ r' \supseteq r * a \notin s * a \notin h * \mathit{payload}(a) = v * \\ \quad \mathit{origin}(a) = i * a \in \text{Maximals}(h \cup \{a\}) * \\ \quad a \in \text{Maximum}(s \cup r' \cup \{a\}) * \\ \quad \text{LocSt}(i, \bullet s \cup \{a\}, \circ r') * \text{GlobSt}(h \cup \{a\})\end{array} \right\rangle^{\mathcal{N}}$$

Fig. 9. Logically-atomic specs for get_state and update where $\mathcal{N}$ must contain the global invariant's name.

the result m is not just the number of recorded operations $|s \cup h'|$ but instead takes into account whether each operation is an inc or a dec.

Scaling up to CRDTs via denotations. The idea of tracking operations as opposed to state (Figure 8) can be applied to specifying additional CRDTs in addition to the G-Counter. We just need two additional ingredients: first, abstract away the function that computes the CRDT's current state from the set of received operations (so instead of returning $|s \cup h'|$ in query we want $f(s \cup h')$ for some f). Second, when operations are not naturally commutative (for example, a replicated register that stores the "last" write) CRDTs use causality information to re-introduce commutativity. This is precisely what Burckhardt et al. [2014] do with their notion of *operation contexts* which "include all we need to know about a[n] [...] execution to determine the return value of a given operation" [Burckhardt et al. 2014]; we will use the related notion of CRDT *denotations* from Leijnse et al. [2019]. The definitions below are implicitly parametrized by a given CRDT; specifically by its set of operations Op and states St.

Definition (Events). *The set of* events *is the product* $\text{Event} \triangleq \text{Op} \times \text{VC} \times \mathbb{N}$, *where* VC *is the type of vector clocks and the third component denotes the originating replica id for the event. We lift the partial order of vector clocks to events.*

Definition (Denotations). *A denotation* $[\![\cdot]\!] : \mathcal{P}(\text{Event}) \rightharpoonup \text{St}$ *is a partial function from sets of events to states.*

As an example, the following is the denotation for a *multi-value register* CRDT. A multi-value register stores only concurrent writes; writes that come later in causal order replace earlier ones. The set Op of operations is just $\{\text{write}(z) | z \in \mathbb{Z}\}$.

$$[\![s]\!]_{\text{mv-reg}} = \{(w, vc) | \exists o.(\text{write}(w), vc, o) \in s \wedge vc \in \text{Maximals}(s)\}$$

A nice feature of denotations is that they support specifying higher-order CRDT *combinators*. For example, given denotations A and B, we can form their product (another denotation) $A \times B$, defined in Section 6.

We can give specifications for OpLib's get_state and update functions that are parametric on the denotations of the CRDT being implemented. These are shown in Figure 9.

GetStateSpec. We use the get_state() function to query the CRDT's state. To verify a call to get_state(), we need to provide the local state resource $\text{LocSt}(i, \bullet s, \circ h)$. When the call completes,

we get back $\mathsf{LocSt}(i, \bullet s, \circ h')$ for some $h' \supseteq h$. That is, we now logically know that the CRDT has received additional remote operations (namely $h' \setminus h$), and that the local operations have not changed (because we were holding the local resource, of which there is only one copy per replica). The return value v of get_state is *coherent* with a logical representation of the state w; this is given by the predicate $\mathsf{StCoh}(w, v)$. We do this because the logical version of the state w might offer a "cleaner" representation of the state that is not polluted by the idiosyncrasies of AnerisLang's design, of which v is a value. For example, w might be a triple while AnerisLang only supports pairs, so w's encoding of v uses nested pairs. Finally, we know that the (logical version of the) return value is the denotation of the observed operations: $[\![s \cup h']\!] = w$.

UpdateSpec. To update the CRDT, we call update(v), where v is some operation.[14] As a pre-condition, we must provide the local and global state resources, $\mathsf{LocSt}(i, \bullet s, \circ r)$ and $\mathsf{GlobSt}(h)$, respectively. The update function returns unit. We get back updated resources $\mathsf{LocSt}(i, \bullet s \cup \{a\}, \circ r')$ and $\mathsf{GlobSt}(h \cup \{a\})$; the latter is because around the linearization point exactly one event has been added to the entire system, namely the new event a containing the operation v. This new event originates at node i, and is maximal with respect to all other events in h, and the maximum of the (local) events in $s \cup r' \cup \{a\}$: this is just like in the broadcast spec in Figure 4. The new local resource $\mathsf{LocSt}(i, \bullet s \cup \{a\}, \circ r')$ indicates that we are now aware of exactly one additional local event a, as well as zero or more remote events $r' \supseteq r$. Finally, $a \notin h$, indicating that every update generates a new event.

Labelled-transition systems. We have seen that denotations provide a high-level specification of a CRDT. The problem, however, is that denotations are too high-level. Specifically, the denotation has access to the entire set of operations performed on the data type, whereas in reality operations arrive one at a time (either from remote updates or due to local function calls). The solution is to give a second, lower-level specification for CRDTs, one that is closer to the running program. We do so using labelled-transition systems (LTS). Our LTS is a tuple $(\mathsf{St}, \mathsf{Event}, \rightarrow, \sigma_0)$ containing the set St of (CRDT) states, the set Event of events which serve as labels (recall that events contain operations plus causality metadata), a (partial) transition function $\rightarrow : \mathsf{St} \times \mathsf{Op} \rightharpoonup \mathsf{St}$, and an initial state σ_0.

Figure 10 shows a sample LTS for a multi-value register. A register state St is a set of pairs $\{(z, t)\}$ containing a value z written to the register together with a timestamp t (a vector clock) of when the write occurred. The transition labels Event are triples $(\mathsf{write}(z), t, r)$ containing a value z written, its timestamp t, and a replica id r of the process that issued the write. The transition relation $\mathsf{st} \xrightarrow{\mathsf{ev}} \mathsf{st}'$ is set up such that from a state st and given an event ev we can move to st′ if st′ consists of ev plus all elements of st that happened *concurrently* with ev. Finally, the initial state σ_0 is the empty set. Notice we assumed the new event ev does *not* happen before any of the writes already in st (that is, we assumed that $\forall e.e \in \mathsf{st} \implies ev \geq e$). This assumption is justified because OpLib is implemented using RcbLib, so we can assume that an operation's causal dependencies are delivered before the operation itself is, so that if $ev < e$ for some $e \in \mathsf{st}$, then $\mathsf{ev} \in \mathsf{st}$ (a contradiction).

We integrate labelled-transition systems into OpLib specs by defining *coherence* properties between (a) a denotation and the corresponding LTS and (b) the LTS and the effect function supplied by the CRDT implementor. The coherence properties are shown in Figure 11.

The coherence between denotation and its LTS is given by two requirements. First, the denotation of the empty set of events should be the initial LTS state σ_0. Second, if $[\![s]\!] = p$ and p steps to p' through a transition labelled e, we must have $[\![s \cup e]\!] = p'$. This last implication is weakened to

[14] Notice when the user calls update they do not know what vector clock will be assigned to the operation; that happens internally once RCB broadcasts the message.

$$
\begin{aligned}
\mathsf{VC} &= \text{representation of vector clocks} \\
\mathsf{RepID} &= \mathbb{N}\ (\text{replica ids}) \\
\mathsf{St} &= \mathcal{P}(\mathbb{Z} \times \mathsf{VC}) \\
\mathsf{Event} &= \{\mathsf{write}(z) | z \in Z\} \times \mathsf{VC} \times \mathsf{RepID} \\
\mathsf{payload}(\mathsf{write}(z), _, _) &= z \\
\mathsf{orig}(_, _, r) &= r \\
\longrightarrow &= \{(\mathsf{st}, \mathsf{ev}, \mathsf{st}') | \mathsf{st}' = (\mathsf{payload}(\mathsf{ev}), \mathsf{orig}(\mathsf{ev})) \cup \mathsf{filter}(\lambda e.e \geq \mathsf{ev})\mathsf{ev}\} \\
\sigma_0 &= \emptyset
\end{aligned}
$$

Fig. 10. Labelled-transition system for a multi-value register

Validity of new messages

$$
\begin{aligned}
\mathsf{Valid}(s, e) &\triangleq e \notin s \wedge e \in \mathsf{Maximals}(s \cup \{e\}) \wedge \mathsf{EventsExt}(s \cup \{e\}) \wedge \mathsf{EventsTotal}(s \cup \{e\}) \\
\mathsf{EventsExt}(s) &\triangleq \forall e\, e'.e \in s \wedge e' \in s \wedge \mathsf{vc}(e) = \mathsf{vc}(e') \implies e = e' \\
\mathsf{EventsTotal}(s) &\triangleq \forall e\, e'.e \in s \wedge e' \in s \wedge \mathit{origin}(e) = \mathit{origin}(e') \wedge e \neq e' \implies e < e' \vee e > e'
\end{aligned}
$$

Coherence of denotation and LTS

$$
\begin{aligned}
&[\![\emptyset]\!] = \sigma_0 \\
&\forall s\, p\, e\, p'.\mathsf{Valid}(s, e) \wedge \\
&\quad [\![s]\!] = p \wedge p \xrightarrow{e} p' \implies [\![s \cup e]\!] = p'
\end{aligned}
$$

Coherence of LTS and effect function

$$
\begin{aligned}
&\textsc{EffectSpec} \\
&\{\mathsf{StCoh}(s, st) \wedge \mathsf{EvCoh}(e, ev) \wedge [\![S]\!] = s \wedge \mathsf{Valid}(S, e) \quad\} \\
&\quad \langle ip_i;\ \mathsf{effect}(\mathsf{ev}, \mathsf{st})\rangle \\
&\{st'.\ \exists s'.\ \mathsf{StCoh}(s', st') \wedge s \xrightarrow{e} s' \quad\}
\end{aligned}
$$

Fig. 11. Coherence properties relating the denotation, labelled-transition system, and effect function

hold only for new messages e that are *valid* with respect to the set of existing events s, written $\mathsf{Valid}(s, e)$.

The validity predicate encodes assumptions we can make about arriving messages because of guarantees provided by causal broadcast. That is, if $\mathsf{Valid}(s, e)$ holds, then $e \notin s$ (there are no duplicates), $e \in \mathsf{Maximals}(s \cup \{e\})$ (no already-delivered message causally depends on e), $\mathsf{EventsExt}(s \cup \{e\})$ (vector clocks uniquely identify messages) and $\mathsf{EventsTotal}(s \cup \{e\})$ (messages originating at the same replica can be totally ordered).

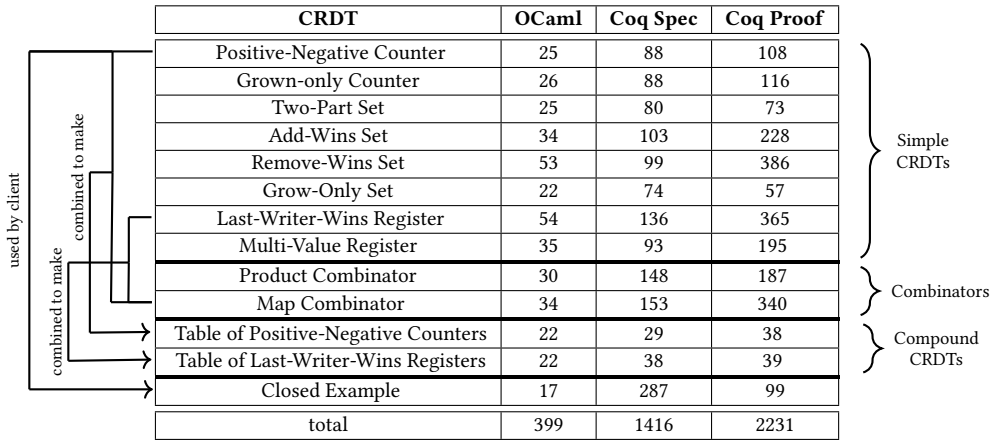
Finally, coherence between the LTS and the effect function is specified via a Hoare triple. [15] The spec says that if we are to execute $\mathsf{effect}(\mathsf{ev}, \mathsf{st})$, then we must know that ev and st are coherent with their logical counterparts s and e, respectively. Additionally, there must be some set of events S such that $[\![S]\!] = s$ and e must be a valid new message with respect to S (so $\mathsf{Valid}(S, e)$). If that is the case, then if effect terminates it will return a new physical state st' such that $s \xrightarrow{e} s'$, where s' is the logical view of st'. That is, the spec says that if we step from st to st' via ev using effect in the physical world, then we can step from s to s' via e using the LTS in the logical world.

[15] Notice that, unlike the spec for update and get_state, the effect spec is given by a regular Hoare triple, as opposed to a logically-atomic triple. This is because effect only manipulates pure propositions and does not require any exclusive resources that need to be stored in invariants.

Table 2. Library metrics (lines of code)

Library	OCaml	Coq Spec	Coq Proof
RcbLib	196	2151	2703
OpLib	86	1352	2224
total	282	3503	4927

Table 3. CRDTs implemented on top of OpLib (lines of code)



CRDT	OCaml	Coq Spec	Coq Proof	
Positive-Negative Counter	25	88	108	Simple CRDTs
Grown-only Counter	26	88	116	
Two-Part Set	25	80	73	
Add-Wins Set	34	103	228	
Remove-Wins Set	53	99	386	
Grow-Only Set	22	74	57	
Last-Writer-Wins Register	54	136	365	
Multi-Value Register	35	93	195	
Product Combinator	30	148	187	Combinators
Map Combinator	34	153	340	
Table of Positive-Negative Counters	22	29	38	Compound CRDTs
Table of Last-Writer-Wins Registers	22	38	39	
Closed Example	17	287	99	
total	399	1416	2231	

Library interface. As shown in Figure 6, a user of OpLib starts by calling oplib_init with a number of arguments. One of them, named crdt in Figure 6, is a pair (init_st, effect) consisting of the data type's initial state and its effect function, respectively. The initial state must be coherent with the LTS's initial state σ_0, so StCoh(σ_0, init_st), and the effect function must satisfy EffectSpec from Figure 11. When oplib_init returns, it gives back a pair of functions (get_state, update) that satisfy GetStateSpec and UpdateSpec from Figure 9.

5.3 Correctness Proof

The core of OpLib's correctness proof is a *lock invariant* [Birkedal and Bizjak 2017] asserting that the CRDT's internal state equals the denotation of the set of operations processed so far. The logical resources needed to enforce this invariant are divided across three areas of responsibility: first, a global invariant tracks the set of messages sent by all replicas, as well as the per-replica delivered messages. This global invariant also asserts that messages are sent via the RCB protocol, allowing us to inherit all resource-related lemmas from Section 4 (e.g. causal delivery). Next, the aforementioned lock invariant also tracks the messages delivered by a specific replica; the messages are divided in two groups: local and remote. Finally, we have the user-resources such as LocSt($i, \bullet s, \circ h$) that are useful for verifying client programs. We use a number of resource algebras, including Timany and Birkedal [2021]'s monotone construction, to carefully coordinate these different logical resources: for example, to prove that ownership of LocSt($i, \bullet s, \circ h$) really does grant precise knowledge of the set of delivered local messages s, but only partial knowledge of the remotely-delivered messages h.

We refer the reader to our Coq development for full details on the proof. Table 2 shows the number of lines of OCaml and Coq code needed to implement and verify both RcbLib and OpLib.

6 IMPLEMENTING CRDTS

In order to put OpLib to test we have implemented twelve CRDTs using this library. These CRDTs consist of eight simple CRDTs, two CRDT combinators, and two compound CRDTs which apply the map combinator to one of the simple CRDTs. Below, we will briefly explain these examples and discuss and summarize what is depicted in Table 3. The relatively low number of lines of code required to implement (in OCaml) and verify the CRDTs enumerated in Figure 3 shows the usefulness and success of our methodology of building CRDTs on top of the RcbLib and OpLib libraries. Moreover, as we will discuss below, the most intricate CRDTs in Table 3, *i.e.*, the last two rows, are those with smallest implementation and verification codes thanks to our compositional approach using CRDT combinators.

Counters. We have implemented two counter variants: Grow-only Counter which can only have non-negative values and can only be incremented, and Positive-Negative Counter which can be both incremented and decremented. These two CRDTs are the simplest examples we have implemented. Part of the size of the Coq code is caused by having to show that the operations are commutative and associative; basic arithmetic facts which nonetheless need to be established formally in Coq.

Sets. The only operation of the Grow-Only Set CRDT allows adding an element to the set. The Add-Wins Set and Remove-Wins Set CRDTs on the other hand support both adding elements and removing elements. They treat the removal operation differently though. The issue with the removal operation is that it causes ambiguity in case of concurrent operations which add and remove the same element. The Add-Wins Set and Remove-Wins Set, as their names indicate, resolve this ambiguity in favor of addition and removal respectively. Despite their apparent similarity these two CRDTs are conceptually different as can be seen in the difference in the number of lines of Coq code required to prove their correctness. The difference is that for the Add-Wins Set CRDT we only remember the additions in the local state. When we receive a removal operation we simply remove any element that was added *strictly* before that removal operation. This makes sense as an addition that is received after a removal can never be affected by it — in worst case, it is an addition concurrent with a removal which by definition wins. On the other hand, in the Remove-Wins Set CRDT we also need to track all remove operations in the local state of each replica as additional operations received after a removal operation can be invalidated by that removal operation. The Two-Part Set CRDT is conceptually simply obtained by gluing two Grow-Only Set CRDTs together. It tracks two sets and operations can add elements to either set. In practice, this CRDT could be obtained by combining the Grow-Only Set CRDT with the Map Combinator as a map with the domain being a fixed set of two elements (see below). However, we chose to implement this CRDT as a yet another simple example from scratch. All set CRDTs are parameterized by the collection of elements that can be stored in sets. In the OCaml code this means that the code is parameterized by a type variable for the type of elements of the set. It is only required that these elements can be serialized as we need to communicate them over the network.

Registers. We have implemented two simple registers: a Multi-Valued Register and a Last-Writer-Wins Register. Just like sets these CRDTs are also parameterized by the collection of values that can be stored in these registers. The difference between these two CRDTs is the way they handle the issue of concurrent write operations. The Multi-Valued Register simply collects all possible values (time-wise maximal write operations) and presents them to the user of the register along with their corresponding time-stamp. The idea is that the user will have the authority to disambiguate the situation. The Last-Writer-Wins Register on the other hand considers the latest write in the set of maximal concurrent writes and considers that to be the valid value of the register. The concurrent nature of the events in our settings means that this method of disambiguation is not always viable.

After all, concurrent events can be observed in different orders by different replicas. To obtain a complete disambiguation strategy the Last-Writer-Wins Register considers the latest write from the replica with the highest replica id to prevail.

Combinators. We have implemented two CRDT combinators: the Product Combinator and the Map Combinator. The Product Combinator takes two CRDTs and constructs a CRDT where the state is the product of two states. The operations of Product CRDT are pairs of operations which take effect component-wise. The Map Combinator is a versatile combinator which takes a CRDT and constructs the CRDT of finite maps, *i.e.*, tables, of that CRDT. The state of the Map CRDT is a map with keys ranging over strings and value ranging over the states of the given CRDT. An operation is a pair of a string, the key to which the operation applies, together with an operation of the underlying CRDT. The map is initially empty. Every time an operation is received for a key that does not exist in the map it is first initialized with the initial state of the given CRDT before the operation is applied to it.

Compound CRDTS. As illustrative examples we have implemented two compound CRDTs. Both of these examples use the Map Combinator. One makes a table of Last-Writer-Wins Registers while the other makes a table of Positive-Negative Counters. The fact that these relatively complicated CRDTs can be constructed and proven correct with very little effort is excellent evidence for the success of our methodology. We obtain full-functional correctness of these essentially databases, albeit with single-column tables, in under 50 lines of Coq code including the boilerplate for including necessary Coq libraries, *etc.*

A concrete closed example. As a minimal smoke test for our OpLib library we prove safety (*i.e.* the program does not crash) of a simple example program. More precisely, using the so-called adequacy theorem of Aneris, we obtain that when this program is executed, as per the operational semantics of AnerisLang, it does not get stuck. This example initializes two replicas of Positive-Negative Counters with initial state 0. The first replica adds 1 to the counter and the second replica adds 2. They both proceed to read the value of the counter after adding to it. Intuitively, we expect the first replica to either read 1 or 3 while the second replica could read 2 or 3; and this is what both replicas *assert* as their last operation. This makes sense as each replica will definitely observe its own performed operation but might or might not have observed the operation performed by the other replica when it reads the counter. The assert command in AnerisLang is designed to evaluate its boolean and ignore it if it evaluates to true (it returns unit) and crash otherwise; hence showing safety of the example does indeed establish that the result each replica obtains when reading the counter is as expected. Intuitively, to establish this property, we simply need to enforce, using an invariant, that the global state (tracked using the proposition $\mathsf{GlobSt}(h)$) has at most two operations in it: an addition of 1 to the counter originating in the first replica and an addition of 2 originating from the second replica. Therefore, each replica by knowing its own local state (tracked using the proposition $\mathsf{LocSt}(i, \bullet s, \circ r)$), and using the relation between the global and local states of CRDTs, can conclude that the value read is the expected one.

7 RELATED WORK

The literature on verification of CRDTs has grown over the years to produce many different approaches. In order to place our work within the mosaic of existing logics and tools, we identify several design criteria that help us build a taxonomy of the sub-field. For each criterion, we propose a concrete question that helps us classify each of the pieces of related work according to the criterion. Table 4 lists our proposed criteria and how to identify whether a paper meets each of them. Table 5 then looks at whether related work meets each criterion. Some works do not fit neatly

Table 4. Classification criteria for CRDT verification techniques

Criterion	Question
Target	Does the technique target op-based or state-based CRDTs? Most verification efforts target one of the two kinds of CRDT, but not both.
Implementation	Does the paper claim to automatically produce executable code?
Convergence	Can the technique prove convergence [Shapiro et al. 2011b]? Convergence means that if two replicas have received the same set of events, then they are in equivalent states.
Eventual Delivery	Can the technique prove eventual delivery [Shapiro et al. 2011b]? Eventual delivery means that an update delivered to a correct replica eventually reaches all correct replicas. This is a liveness property.
Causality	If required, does the paper show that messages are delivered in causal order? The alternative is either that causal delivery is not required for the specific CRDT implemented, or it is required but is then assumed.
Functional Correctness	Can the technique prove functional correctness? That is, are there specifications that show how the outputs of CRDT operations depend on their inputs?
Modularity	Does the paper show an example of a client that uses the CRDT's specification to verify some property? For example, given a G-Counter CRDT can we show that if a replica calls inc twice the counter's value is at least two?
Mechanization	Are the proofs mechanized in a proof assistant?

Table 5. Comparison of different CRDT verification techniques. "Event. Del." stands for eventual delivery, and "F.C." for functional correctness.

Paper	Target	Implementation	Convergence	Event. Del.	Causality	F.C.	Modularity	Mechanization
Burckhardt et al. [2014]	both		✓			✓		
Zeller et al. [2014]	state		✓		free	✓		✓
Nair et al. [2020]	state		✓[16]		free			
Timany et al. [2021]	state	✓	✓	✓	free	✓	✓	✓
Gomes et al. [2017]	op	✓	✓					✓
Liu et al. [2020]	op	✓	✓		✓	✓		✓
Liang and Feng [2021]	op		✓			✓	✓	
Nagar and Jagannathan [2019]	op		✓					✓
this work	op	✓	✓		✓	✓	✓	✓

in their assigned classes, nor do we argue that our choice of questions is canonical. We nevertheless think that posing concrete questions leads to a classification that is imperfect but useful.

We structure our discussion of related work around Table 5. Most techniques target either state-based CRDTs or op-based CRDTs, but not both. The exception is Burckhardt et al. [2014], which focuses mostly on specifying both kinds of CRDTs via denotations, but not on verification.

Verification of state-based CRDTs. As explained in Burckhardt et al. [2014] state-based approaches guarantee causal delivery "for free". This is because communicating an entire state is (logically) equivalent to sending an operation together with all its causal dependencies.

The only modular state-based approach that we are aware of is Timany et al. [2021]. This is also the only related work that proves eventual delivery. Like us, Timany et al. [2021] use the Aneris separation logic; however, unlike us they only verify one example CRDT (a G-Counter), and their specification style (which tracks states as opposed to sets of operations) is less expressive than ours (see Section 5). To prove liveness, Timany et al. [2021] develop an extension to the Iris program logic framework called Trillium, which allows them to show that the CRDT implementation refines a state-transition system. It would be interesting to restructure our development to use Trillium, since we already show that our CRDT implementations implement a labelled-transition system. Finally, Timany et al. [2021] focus on (one) state-based CRDT, whereas we verify multiple op-based CRDTs (see Section 8 for a discussion of how our approach could be extended to the state-based setting).

[16]In addition to convergence, Nair et al. [2020] prove other safety properties for specific CRDTs.

Verification of op-based CRDTs. Liu et al. [2020] extend Liquid Haskell [Vazou et al. 2014] by annotating typeclass declarations with refinement types. Their system can later typecheck typeclass instances against the declarations. As a case study, they define a typeclass for op-based CRDTs and implement several instances, including a map combinator similar to ours. Instances of the CRDT class enjoy a *strong convergence* property that says that certain allowed permutations of a set of operations lead to the same final state. Additionally, they show functional correctness of their multi-set implementation by a simulation argument with respect to an abstract denotational specification (similarly to how we use denotations). They design their CRDTs so that they do not have to assume causal delivery, although in the process they do end up implementing parts of a causal broadcast algorithm (e.g. a delay queue).

The main difference between Liu et al. [2020] and our work is how modular the approaches are. In Liu et al. [2020] there does not seem to be a way to use strong convergence to verify a client program that uses a CRDT. [17] By contrast, as shown in Section 6 we can use our separation logic resources that track local and global states not only to show convergence and functional correctness, but also to verify clients. Additionally, we were able to verify causal broadcast as a general purpose library which is then re-used by our CRDT library. In their work, only the "business-logic" part of the CRDT is verified: that is their CRDTs are purely-functional data structures that are unaware of the existence of other replicas. By contrast, we verify not only the purely-functional part of a CRDT, but also all of its logic all the way through to network operations.

Liang and Feng [2021] introduce the first technique that produces modular specifications for op-based CRDTs. Specifically, they strengthen SEC to arrive at a trace property called *Abstract Converging Consistency* (ACC), which combines SEC with functional correctness. Functional correctness is obtained by relating a concrete CRDT model Π to its abstract counterpart Γ. In proving the relation, one is allowed to re-order certain abstract (commutative) operations that satisfy an *arbitration* relation $\bowtie$. Once we prove ACC, an *abstraction* theorem gives us contextual refinement: meaning that in every program we can substitute the concrete CRDT by the abstract one (its spec) and still obtain the same results. The paper then introduces a rely-guarantee style logic to prove specification for clients using the CRDT.

Our work differs from Liang and Feng [2021]'s in several aspects. First, their CRDTs, including the concrete variants, are closer to what we would call specifications and not executable implementations. This is because they represent CRDTs as collections of functions that go from state to state via operations (this is very similar to our LTS-based models). In contrast, our CRDT implementations are written in OCaml and so must deal with many details associated with running code: e.g. message serialization, network sockets, node-local concurrency, and mutation. Second, when proving functional correctness of a client in their system one proves a judgment of the form $\vdash \{P\}$ with $(\Gamma, \bowtie)$ do $C_1 || \ldots || C_n \{Q\}$. That is, the existence of the CRDT is baked into the top-level term that one reasons about, and the CRDT Γ is distinguished from the clients C_i. By contrast, in our setting the CRDT and client code are both written in AnerisLang, and our reasoning principles come in the form of standard separation-logic resources (e.g. $\mathsf{LocSt}(i, \bullet s, \circ h)$). We expect that our approach makes it easier to re-use a verified CRDT as a component of a larger system; for example, we were able to create and use CRDT combinators.

Nagar and Jagannathan [2019] present a framework for automated verification of op-based CRDTs, as well as multiple examples of verified CRDTs. Importantly, their technique is parametric on the (axiomatized) consistency model afforded by the underlying communication protocol, so

[17] This is backed by the fact that they do not verify the client applications (a text editor and event calendar) that use their CRDTs.

that the same CRDT implementation can be verified under e.g. eventual consistency and causal consistency.

There are multiple differences between this paper and our work. Their tool targets automated verification of high-level CRDT implementations (labelled-transition systems), while we do interactive verification of low-level OCaml-like code (including concurrency, mutation, higher-order funtions, serialization of network messages, etc.). Additionally, the property they verify is convergence, while we verify convergence and functional correctness. Another difference is that their technique is parametric on a consistency model. By contrast, we fix causal consistency as the guarantee of our broadcast protocol. However, instead of axiomatizing the consistency guarantees, we implement and verify a general purpose library for causal broadcast. Finally, their verification engine is built specifically for (automated) verification of op-based CRDTs. By contrast, we conduct our proofs in a vanilla distributed separation logic (Aneris), using standard features like invariants and ghost state. This means that we are able to compose our proofs and specs with other verification efforts that do not target CRDTs. For example, we composed the proofs of our CRDT library with those of the causal broadcast library.

Verified causally-consistent key-value store. Gondelman et al. [2021] implement and verify a causally-consistent distributed key-value store, also using Aneris. Even though the term "CRDT" does not appear in their paper, they implement and model causal delivery of key-value store operations, and more generally their key-value store is very close to being a CRDT. It is not because it violates SEC: in certain execution traces, we can end up with replicas that have received the same set of writes, yet the same key is mapped to different values. This is because their tie-breaking mechanism for concurrent writes is to take the write that arrives later, which is sensitive to network delays. Additionally, their db replicas do not re-transmit dropped messages, and they do not re-broadcast messages. As mentioned in Section 4 we adapt Gondelman et al. [2021]'s modelling of causality in separation logic so that it is applicable to a general-purpose RCB protocol. In fact, our table-of-registers example from Table 3 is also a key-value store where the above reliability issues are addressed. Our work can be then seen as generalizing the approach in their paper to apply to a wide range of CRDTs, as opposed to a bespoke key-value store.

Verified causal broadcast. Redmond et al. [2022] implement and verify a library for causal broadcast in Liquid Haskell. Specifically, they define a predicate on library states called *process local causal delivery* (PLCD). Their main result is a theorem stating that PLCD is preserved by arbitrary sequences of the three library operation: receive, deliver, and broadcast. They then shows that if every process satisfies PLCD then the entire system satisfies a (global) definition of causal delivery. This is done at the model level, with the entire system modelled as an STS. The paper then uses the verified library to build an (unverified) replicated key-value store. The store code is responsible for network operations, concurrent access to library state via the STM monad, and (de)serialisation of messages. The authors evaluate the key-value store by load-testing it with multiple replicas and clients.

The main difference between Redmond et al. [2022] and our work is the scope of the verified components. Redmond et al. [2022] verify that applying a sequence of library operations starting from an initial empty state preserves PLCD. These operations are pure functions without side effects; network operations and concurrency are instead handled by an unverified library client (their key-value store). By contrast, we verify both the state manipulation functions and also their wrapper code that performs network operations, as well as concurrency control. Another key difference is that in our work we use the verified causal broadcast library as a building block over which to implement and verify our CRDT library, showing that our approach is modular. Redmond et al. [2022] also implement clients on top of their causal broadcast library, but their clients are

unverified. Finally, Redmond et al. [2022] present a performance evaluation of their causal broadcast library, whereas we have not evaluated ours.

8 CONCLUSIONS

We have verified implementations of multiple op-based CRDTs in separation logic. We structured our development as a collection of libraries. First, we verified an RcbLib library for reliable causal broadcast. On top of RcbLib we then verified an OpLib library for building op-based CRDTs. CRDT implementers can use OpLib to specify their CRDTs as purely-functional data structures, without having to worry about low-level implementation details such as network operations and concurrency control. Finally, using OpLib we verified multiple CRDT instances: some are naturally commutative, while others use causality information and metadata to make operations commutative. That we were able to handle different kinds of CRDTs, including higher-order combinators, shows the applicability of our technique to a variety of scenarios. Our approach both can verify realistic implementations (as opposed to high-level protocols) and is modular (we can verify components in isolation and put their proofs back together to obtain verified stacks of components).

Future work: state-based CRDTs. A natural question is whether our techniques could be adapted to verify state-based CRDTs. More precisely, whether local and global resources can track operations in that setting even though the entire CRDT state is sent between replicas, as opposed to sending one operation at a time. We think the answer is yes; the insight is that when two CRDT states, which are lattice elements, are merged by taking their least upper bound, the operations that generated those states can also be merged. [18] That is, logically we can also take the least upper bound in the powerset lattice of operations, so that if we are in a state $\mathsf{LocSt}(i, \bullet\, s, \circ\, h)$ and a foreign replica sends their state that resulted from operations coming from a set q, then we can update our state to $\mathsf{LocSt}(i, \bullet\, s, \circ\, h \cup q')$, where $q' = \{e \in q | origin(e) \neq i\}$. The goal of this line of work would be to produce a common specification style for both kinds of CRDTs, so that clients can use a CRDT without worrying about which implementation strategy was used.

Retrospective on our proofs. The RcbLib project was sparked by the observation that we can disentagle the logical account of causality in Gondelman et al.'s work from the application domain of their paper (a distributed key-value store). The main challenge was then to use a similar flavour of separation logic resources to verify a general-purpose reliable causal broadcast library. In particular, to provide reliability RcbLib deploys a number of techniques (sequence ids, acknowledgements, retransmissions) not present in Gondelman et al.'s work that complicate the proofs.

The challenge in verifying OpLib was in adapting the notion of CRDT denotation to a separation logic setting. In particular, it was challenging to construct the $\mathsf{LocSt}(i, \bullet\, s, \circ\, r)$ resource that tracks local events precisely and remote events loosely. From $\mathsf{LocSt}(i, \bullet\, s, \circ\, r)$ we can connect the return value of get_state to the denotation of the set of delivered local events.

What worked well: our Coq formalization makes extensive use of typeclasses to provide clean interfaces between each of the components (e.g. there are typeclasses for denotations, the LTS model, and the local and global resources). Additionally, in order to make the verification of our examples manageable we had to structure OpLib so that it would abstract away as much as possible from a CRDT implementation. This worked well; for instance, one of the authors verified most of the example CRDTs without having prior involvement in the verification of OpLib (they relied solely on OpLib's specification). Once the OpLib interface was ironed out, we were able to verify OpLib and its clients in parallel.

[18] This idea appeared in Burckhardt et al. [2014]; the challenge would be to adapt it to separation logic so we can verify implementations modularly.

What did not work as well: it was more challenging than we expected to connect OpLib's logical resources to their RcbLib counterparts. This is necessary so OpLib can inherit all the resource laws that RcbLib provides (e.g. so we can rely on causality when reasoning about CRDTs). Establishing the connection was trickier than we thought because RcbLib's events have untyped payloads (we make no assumptions about the contents of broadcast messages), while OpLib uses typed payloads (for a specific CRDT we know the shape of its operations). This kind of impedance mismatch required proving many additional auxiliary lemmas, thus increasing OpLib's proof effort.

DATA AVAILABILITY STATEMENT

The AnerisLang implementation of the libraries and examples described in this paper, as well as their safety proofs mechanized in Coq using Aneris, can be found in Nieto et al. [2022].

ACKNOWLEDGMENTS

This work was supported in part by a Villum Investigator grant (no. 25804), Center for Basic Research in Program Verification (CPV), from the VILLUM Foundation.

REFERENCES

Mustaque Ahamad, Gil Neiger, James E. Burns, Prince Kohli, and Phillip W. Hutto. 1995. Causal Memory: Definitions, Implementation, and Programming. *Distributed Comput.* 9, 1 (1995), 37–49. https://doi.org/10.1007/BF01784241

Peter Bailis, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Bolt-on causal consistency. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*. 761–772. https://doi.org/10.1145/2463676.2465279

Carlos Baquero, Paulo Sérgio Almeida, and Ali Shoker. 2014. Making operation-based CRDTs operation-based. In *Proceedings of the First Workshop on the Principles and Practice of Eventual Consistency, PaPEC@EuroSys 2014, April 13, 2014, Amsterdam, The Netherlands*, Marc Shapiro (Ed.). ACM, 7:1–7:2. https://doi.org/10.1145/2596631.2596632

Lars Birkedal and Aleš Bizjak. 2017. Lecture Notes on Iris: Higher-Order Concurrent Separation Log. (2017). http://iris-project.org/tutorial-pdfs/iris-lecture-notes.pdf

Kenneth Birman, Andre Schiper, and Pat Stephenson. 1991. Lightweight Causal and Atomic Group Multicast. *ACM Transactions on Computer Systems (TOCS)* 9, 3 (1991), 272–314. https://doi.org/10.1145/128738.128742

Sebastian Burckhardt, Alexey Gotsman, Hongseok Yang, and Marek Zawirski. 2014. Replicated Data Types: Specification, Verification, Optimality. In *41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2014)*. ACM, 271–284. https://doi.org/10.1145/2535838.2535848

Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. 2011. *Introduction to Reliable and Secure Distributed Programming*. Springer Science & Business Media, Chapter 3.

Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. 2019. Verifying concurrent, crash-safe systems with Perennial. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*, Tim Brecht and Carey Williamson (Eds.). ACM, 243–258. https://doi.org/10.1145/3341301.3359632

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Michael Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. 2008. Bigtable: A Distributed Storage System for Structured Data. *ACM Trans. Comput. Syst.* 26, 2 (2008), 4:1–4:26. https://doi.org/10.1145/1365815.1365816

Kristina Chodorow and Michael Dirolf. 2010. *MongoDB - The Definitive Guide: Powerful and Scalable Data Storage*. O'Reilly.

Colin J Fidge. 1987. Timestamps in Message-Passing Systems That Preserve the Partial Ordering. (1987).

Seth Gilbert and Nancy A. Lynch. 2002. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News* 33, 2 (2002), 51–59. https://doi.org/10.1145/564585.564601

Victor B. F. Gomes, Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. 2017. Verifying Strong Eventual Consistency in Distributed Systems. *Proc. ACM Program. Lang.* 1, OOPSLA (2017), 109:1–109:28. https://doi.org/10.1145/3133933

Léon Gondelman, Simon Oddershede Gregersen, Abel Nieto, Amin Timany, and Lars Birkedal. 2021. Distributed Causal Memory: Modular Specification and Verification in Higher-Order Distributed Separation Logic. *Proc. ACM Program. Lang.* 5, POPL (2021), 1–29. https://doi.org/10.1145/3434323

Maurice Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (1990), 463–492. https://doi.org/10.1145/78969.78972

Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20.

https://doi.org/10.1017/S0956796818000151

Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. 637–650. https://doi.org/10.1145/2676726.2676980

Morten Krogh-Jespersen, Amin Timany, Marit Edna Ohlenbusch, Simon Oddershede Gregersen, and Lars Birkedal. 2020. Aneris: A Mechanised Logic for Modular Reasoning about Distributed Systems. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*. 336–365. https://doi.org/10.1007/978-3-030-44914-8_13

Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM* 21, 7 (1978), 558–565. https://doi.org/10.1145/359545.359563

Adriaan Leijnse, Paulo Sérgio Almeida, and Carlos Baquero. 2019. Higher-Order Patterns in Replicated Data Types. In *PaPoC@EuroSys*. ACM, 5:1–5:6. https://doi.org/10.1145/3301419.3323971

Hongjin Liang and Xinyu Feng. 2021. Abstraction for Conflict-Free Replicated Data Types. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 20211*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 636–650. https://doi.org/10.1145/3453483.3454067

Yiyun Liu, James Parker, Patrick Redmond, Lindsey Kuper, Michael Hicks, and Niki Vazou. 2020. Verifying Replicated Data Types with Typeclass Refinements in Liquid Haskell. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 216:1–216:30. https://doi.org/10.1145/3428284

Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2011. Don't settle for eventual: scalable causal consistency for wide-area storage with COPS. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles 2011, SOSP 2011, Cascais, Portugal, October 23-26, 2011*. 401–416. https://doi.org/10.1145/2043556.2043593

Friedemann Mattern et al. 1988. *Virtual Time and Global States of Distributed Systems*. Univ., Department of Computer Science.

Kartik Nagar and Suresh Jagannathan. 2019. Automated Parameterized Verification of CRDTs. In *CAV (2) (Lecture Notes in Computer Science, Vol. 11562)*. Springer, 459–477.

Sreeja S. Nair, Gustavo Petri, and Marc Shapiro. 2020. Proving the Safety of Highly-Available Distributed Objects. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 544–571. https://doi.org/10.1007/978-3-030-44914-8_20

Abel Nieto, Léon Gondelman, Alban Reynaud, Amin Timany, and Lars Birkedal. 2022. *Modular Verification of Op-Based CRDTs in Separation Logic (Proof Artifact)*. https://doi.org/10.5281/zenodo.7055010

Patrick Redmond, Gan Shen, Niki Vazou, and Lindsey Kuper. 2022. Verified Causal Broadcast with Liquid Haskell. *arXiv preprint arXiv:2206.14767* (2022). https://doi.org/10.48550/arXiv.2206.14767

A. W. Roscoe. 1996. Intensional Specifications of Security Protocols. In *CSFW*. IEEE Computer Society, 28–38.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011a. *A comprehensive study of Convergent and Commutative Replicated Data Types*. Research Report 7506. INRIA. http://hal.inria.fr/inria-00555588/

Marc Shapiro, Nuno M. Preguiça, Carlos Baquero, and Marek Zawirski. 2011b. Conflict-Free Replicated Data Types. In *Stabilization, Safety, and Security of Distributed Systems - 13th International Symposium, SSS 2011, Grenoble, France, October 10-12, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6976)*, Xavier Défago, Franck Petit, and Vincent Villain (Eds.). Springer, 386–400. https://doi.org/10.1007/978-3-642-24550-3_29

Swaminathan Sivasubramanian. 2012. Amazon dynamoDB: a seamlessly scalable non-relational database service. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20-24, 2012*. 729–730. https://doi.org/10.1145/2213836.2213945

Andrew S. Tanenbaum and Maarten van Steen. 2007. *Distributed systems - principles and paradigms, 2nd Edition*. Pearson Education.

Amin Timany and Lars Birkedal. 2021. Reasoning about Monotonicity in Separation Logic. In *CPP*. ACM, 91–104. https://doi.org/10.1145/3437992.3439931

Amin Timany, Simon Oddershede Gregersen, Léo Stefanesco, Léon Gondelman, Abel Nieto, and Lars Birkedal. 2021. Trillium: Unifying refinement and higher-order distributed separation logic. *arXiv preprint arXiv:2109.07863* (2021). https://doi.org/10.48550/arXiv.2109.07863

Misha Tyulenev, Andy Schwerin, Asya Kamsky, Randolph Tan, Alyson Cabral, and Jack Mulrow. 2019. Implementation of Cluster-wide Logical Clock and Causal Consistency in MongoDB. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*. 636–650. https://doi.org/10.1145/3299869.3314049

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon L. Peyton Jones. 2014. Refinement Types for Haskell. In *ICFP*. ACM, 269–282. https://doi.org/10.1145/2628136.2628161

Peter Zeller, Annette Bieniusa, and Arnd Poetzsch-Heffter. 2014. Formal Specification and Verification of CRDTs. In *FORTE (Lecture Notes in Computer Science, Vol. 8461)*. Springer, 33–48. https://doi.org/10.1007/978-3-662-43613-4_3